

Graph Neural Networks: Model I

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

<https://ulab-uiuc.github.io/CS512/>

Announcements

- **Reminder:** Writing task due **Sept 17 11:59pm**
 - [Template](#) has been released on Canvas
- Course project:
 - A group of **3 students is expected**. You may start finding teammates on Slack.
 - If you have solid reasons to form a group of 2 or 4, please let our TA know
 - Expect collaborations: Every member will submit a brief *summary of contributions* in final submission
- Topics: any research idea related to graphs. Some references
 - Leverage your insights from **writing tasks, idea proposal, and discussions**
 - We will also suggest some potential project ideas
 - <https://logconference.org/cfp/>, <https://glfrontiers.github.io/2023/cfp/>
 - Focus on ideas that need low resources (Google Colab / laptop / free LLM tools). Proof-of-concept style of experiment will be sufficient

Example Table in the Writing Task

Resource	ToolBench (this work)	APIBench (Patil et al., 2023)	API-Bank (Li et al., 2023a)	ToolAlpaca (Tang et al., 2023)	ToolBench (Xu et al., 2023b)
Real-world API?	✓	✗	✓	✗	✓
Real API Call&Response?	✓	✗	✓	✗	✓
Multi-tool Scenario?	✓	✗	✗	✗	✗
API Retrieval?	✓	✓	✗	✗	✓
Multi-step Reasoning?	✓	✗	✓	✓	✓
Number of tools	3451	3	53	400	8
Number of APIs	16464	1645	53	400	232
Number of Instances	126486	17002	274	3938	2746
Number of Real API Calls	469585	0	568	0	3926
Avg. Reasoning Traces	4.0	1.0	2.1	1.0	5.9

Table 1: A comparison of our ToolBench to notable instruction tuning dataset for tool learning.

		Paper 1
Paper Title		
Paper Link		
LLM Tool		
	Your Answer	
	Your Prompt (Fill in only when using LLMs)	
What is the problem?	LLM Answer (Fill in only when using LLMs)	
	Your Answer	
	Your Prompt (Fill in only when using LLMs)	
Why it is interesting and important?	LLM Answer (Fill in only when using LLMs)	
	Your Answer	
	Your Prompt (Fill in only when using LLMs)	
Why is it hard? (E.g., why do naïve approaches fail?)	LLM Answer (Fill in only when using LLMs)	
	Your Answer	

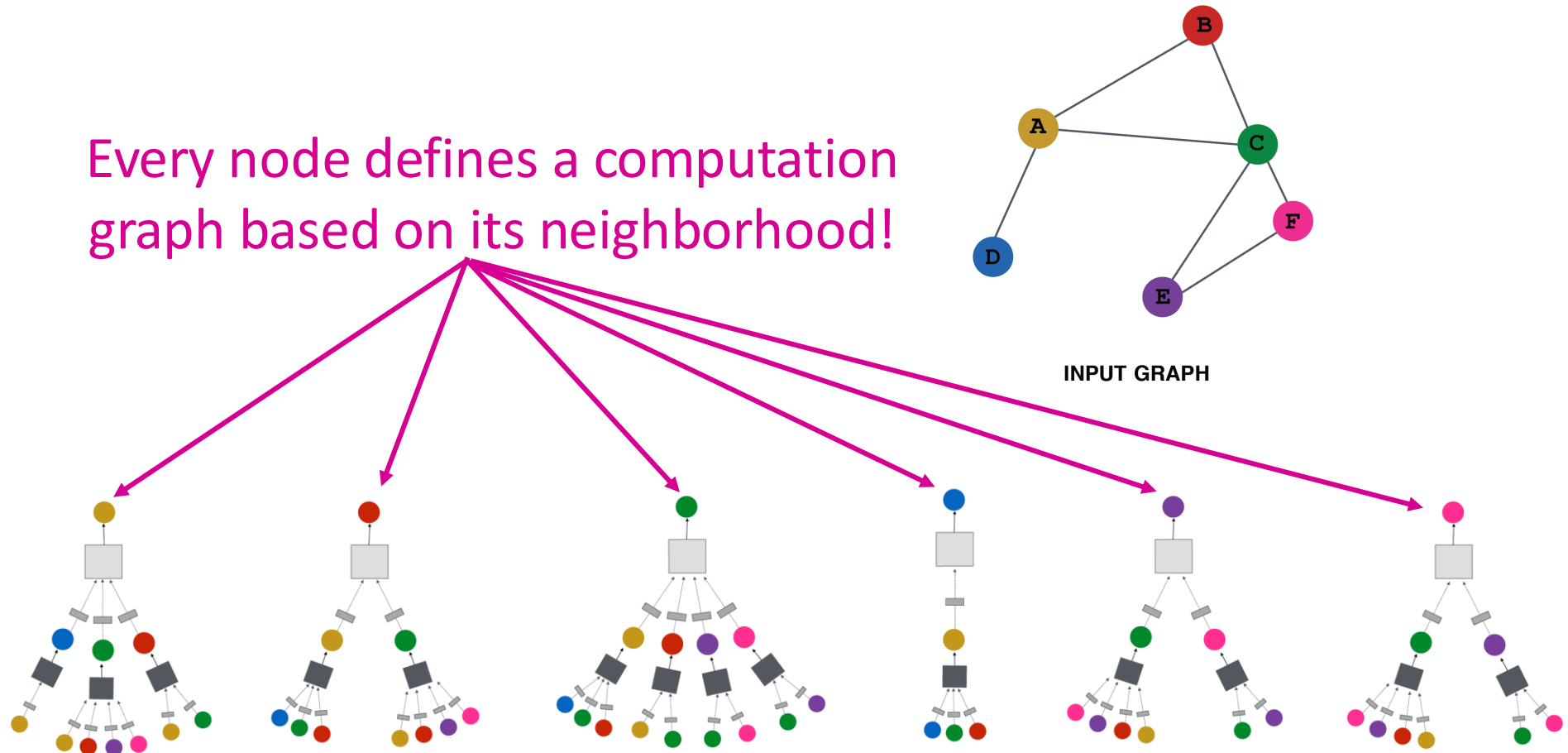
If you didn't use LLM, you don't need to fill (prompt, LLM answer)

Otherwise, you MUST fill it out

Recap: GNN Defines Comp Graph for Each Node

- **Intuition:** Network neighborhood defines a computation graph

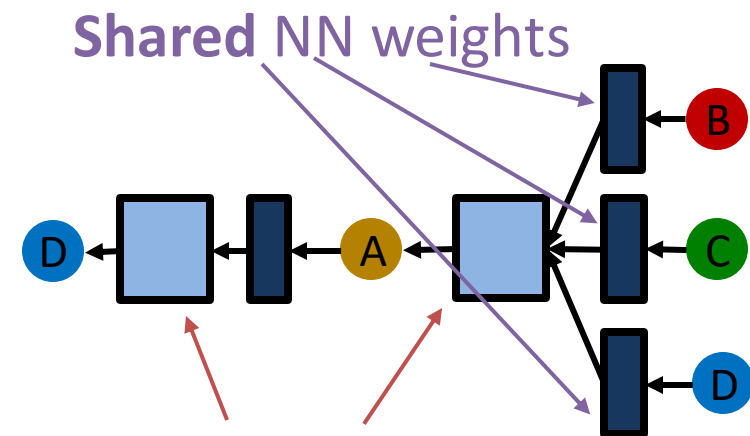
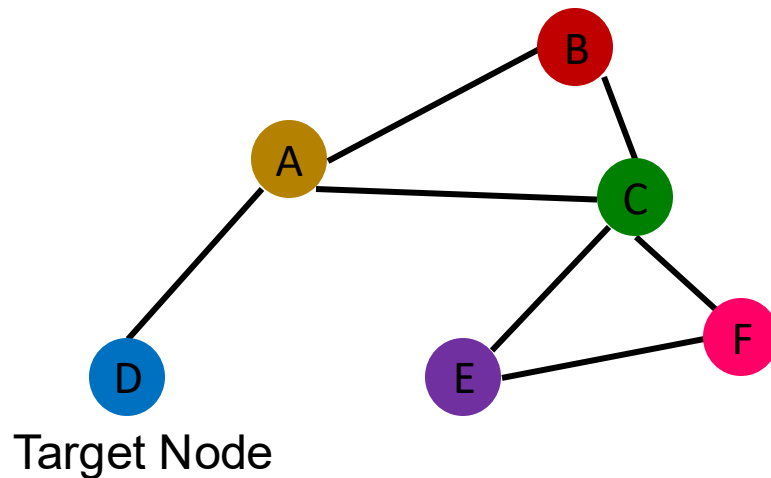
Every node defines a computation graph based on its neighborhood!



Recap: GCN: Invariance and Equivariance

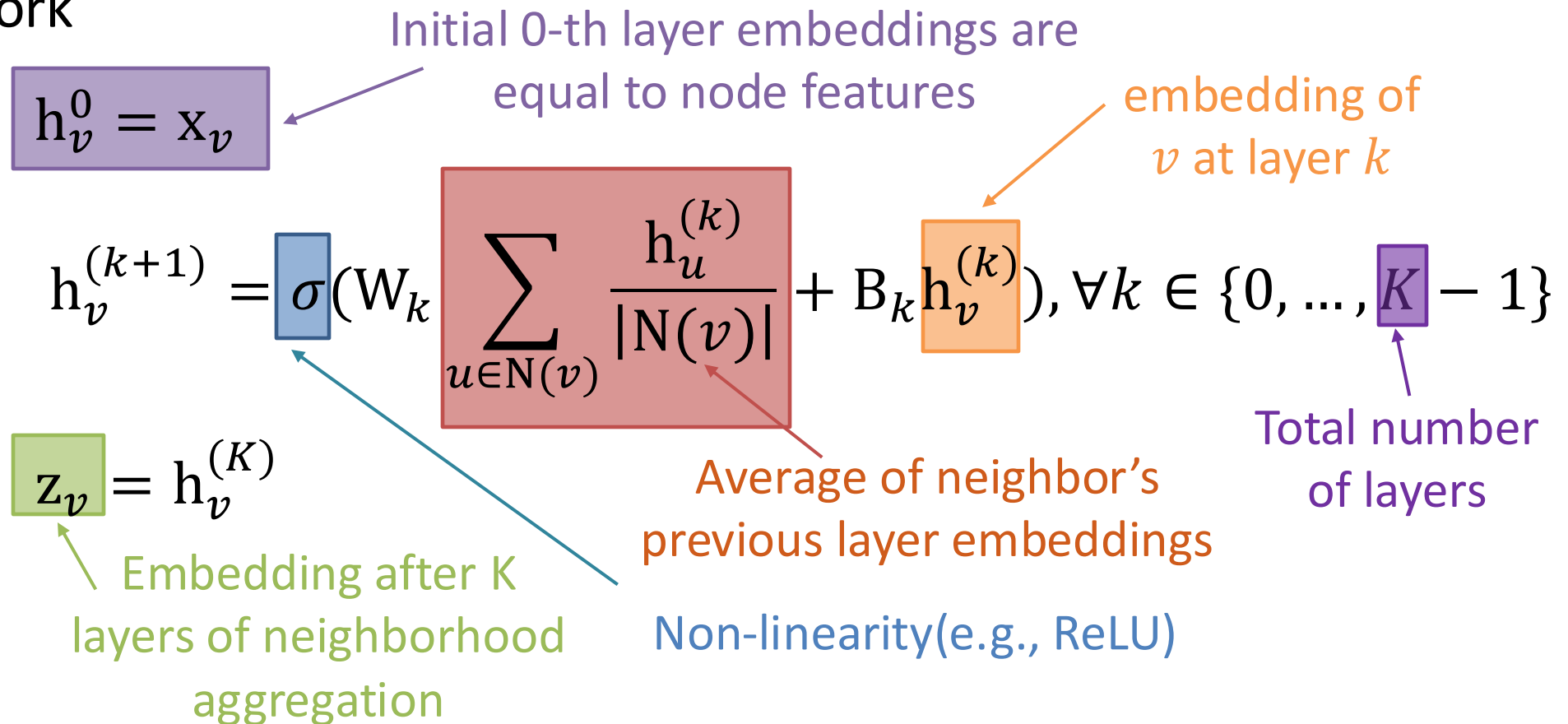
What are the **invariance** and **equivariance** properties for a GCN?

- **Given a node**, the GCN that computes its embedding is **permutation invariant (output one embedding)**



Recap: GCN Encoder

- **Basic approach:** Average neighbor messages and apply a neural network



GCN Matrix Formulation

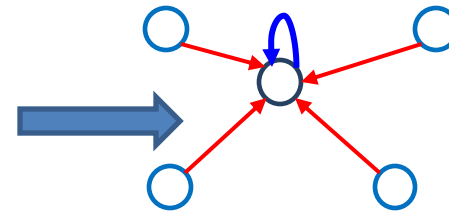
$$h_v^{(k+1)} = \sigma(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}), \forall k \in \{0..K-1\}$$

We have seen GCN update function in **vector form**

- Re-writing GCN update function in **matrix form**:

$$H^{(k+1)} = \sigma(\tilde{A} H^{(k)} W_k^T + H^{(k)} B_k^T)$$

where $\tilde{A} = D^{-1} A$



$$H^{(k)} = [h_1^{(k)} \dots h_{|V|}^{(k)}]^T$$

- Red: neighborhood aggregation Blue: self transformation
- D is a diagonal matrix where $D_{v,v} = \text{Deg}(v) = |N(v)|$
- In practice, this implies that efficient sparse matrix multiplication can be used
- Note:** not all GNNs can be expressed in matrix form, when aggregation function is complex

How to Train A GNN

- Objective: $\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{z}_v))$
 - $\mathbf{y}$: node/edge/graph label
 - $f(\mathbf{z}_v)$ could be node/edge/graph-level prediction head (lecture 2)
 - Θ are trainable GNN weights
 - $\mathcal{L}$ could be L2 loss if $\mathbf{y}$ is real number, or cross entropy loss if $\mathbf{y}$ is categorical
- **Supervised setting:**
 - $\mathbf{y}$ are external labels to graphs
- **Unsupervised/self-supervised setting:**
 - Use the graph structure as the supervision, e.g., similarity based loss function (lecture 3)

GCN Trainable Parameters

Trainable weight matrices
(i.e., what we learn)

$$\begin{aligned} h_v^{(0)} &= x_v \\ h_v^{(k+1)} &= \sigma \left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)} \right), \forall k \in \{0..K-1\} \\ z_v &= h_v^{(K)} \end{aligned}$$

Final node embedding

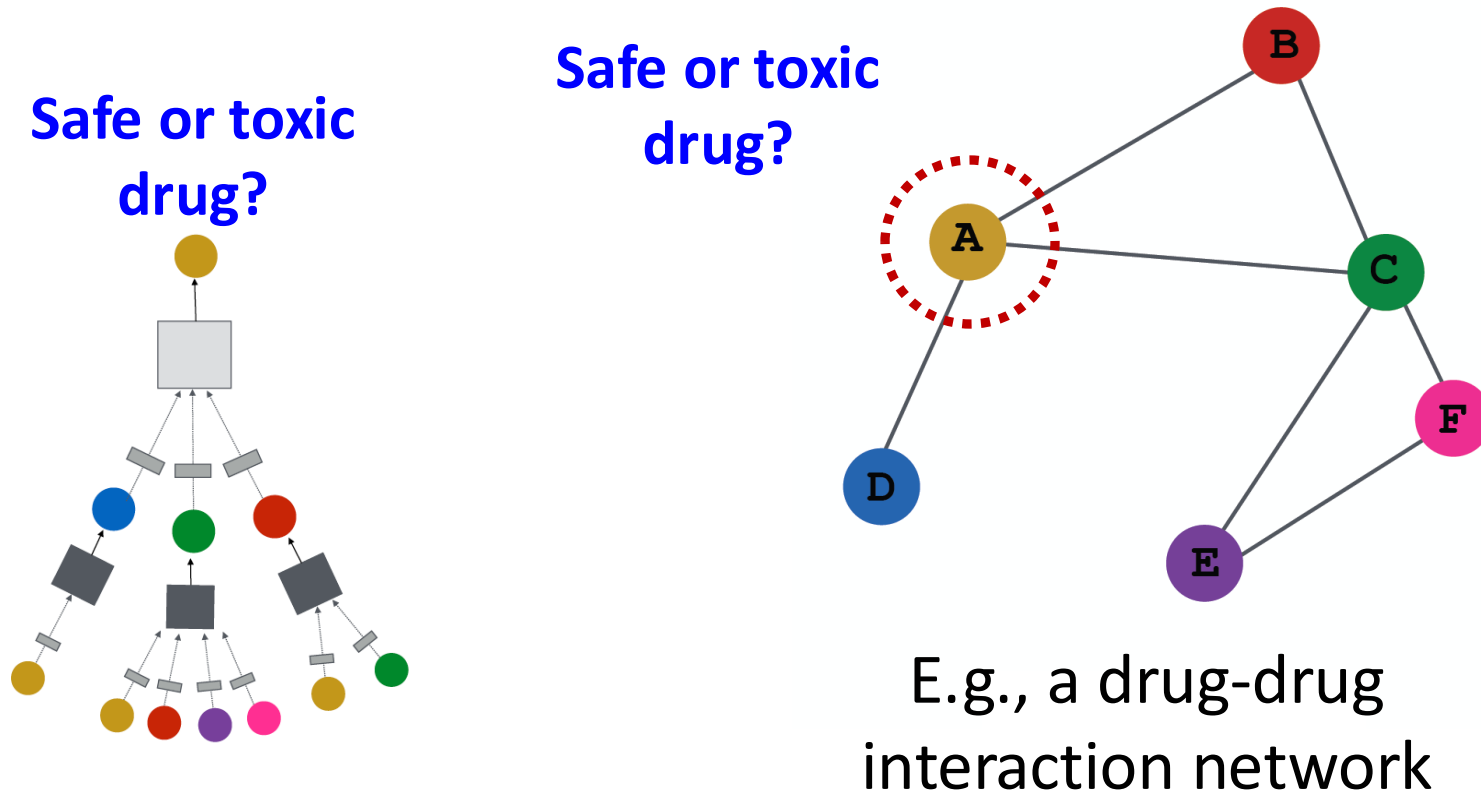
We can feed these **embeddings into any loss function** and run SGD to **train the weight parameters**

h_v^k : the hidden representation of node v at layer k

- W_k : weight matrix for neighborhood aggregation
- B_k : weight matrix for transforming hidden vector of self

Example: Supervised Training

- **Directly train** the model for a supervised task (e.g., node classification)



Example: Supervised Training

Directly train the model for a supervised task (e.g., **node classification**)

- Use cross entropy loss

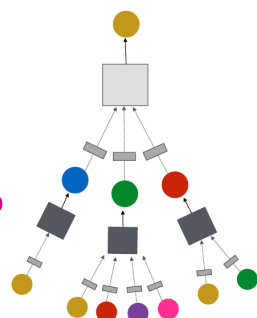
$$\mathcal{L} = - \sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v) \log(1 - \sigma(z_v^T \theta))$$

Encoder output:
node embedding

Classification weights

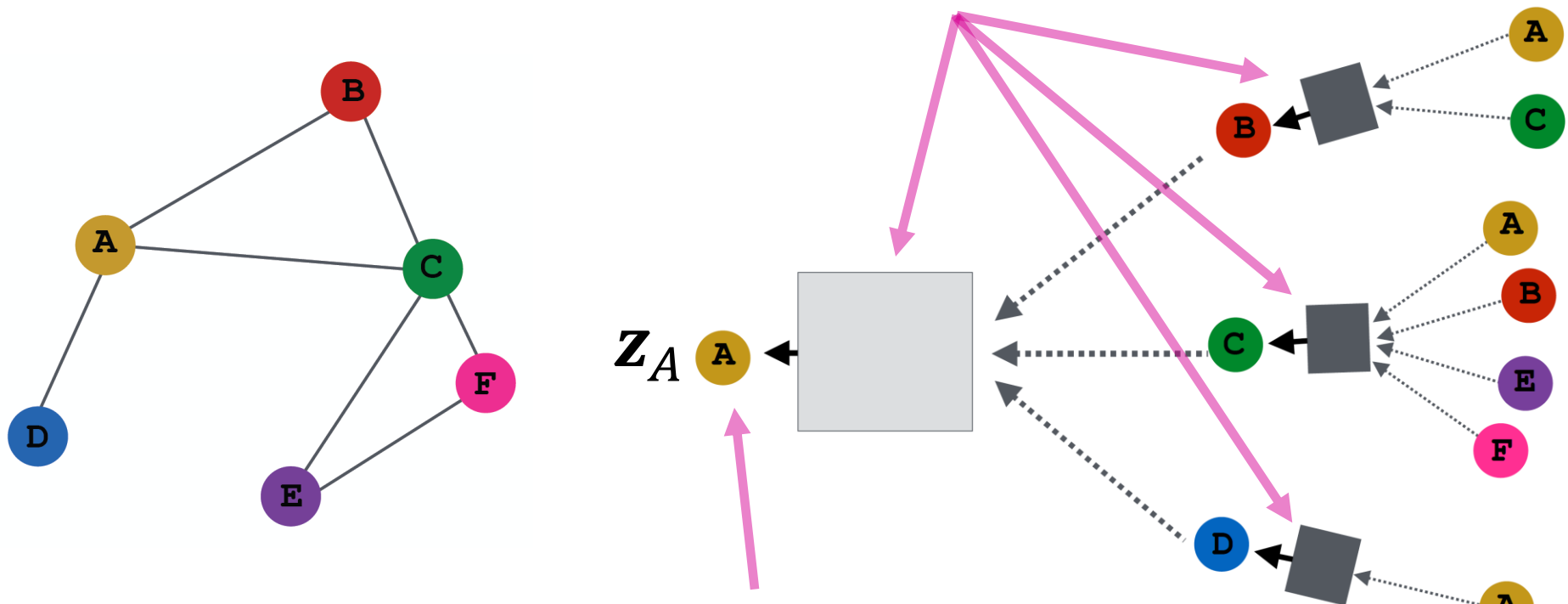
Node class label

Safe or toxic drug?



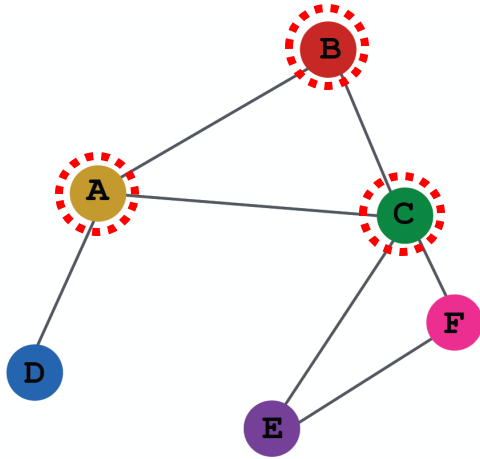
GCN Pipeline: Overview

(1) Define a neighborhood aggregation function



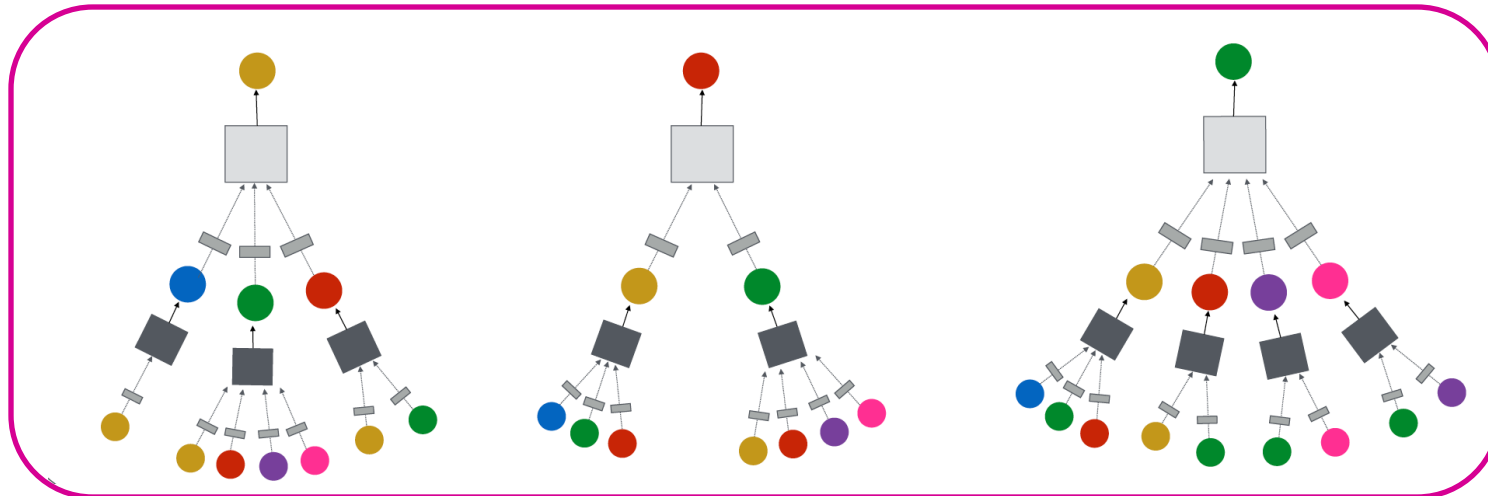
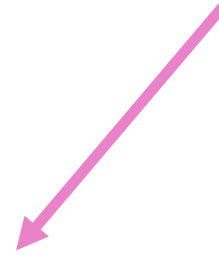
(2) Define a loss function on the embeddings

GCN Pipeline: Overview

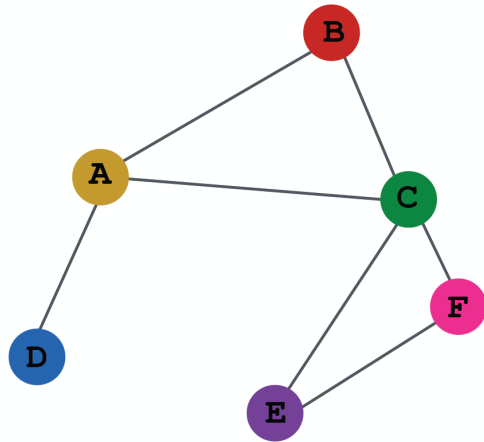


INPUT GRAPH

(3) Train on a set of nodes, i.e.,
a batch of compute graphs



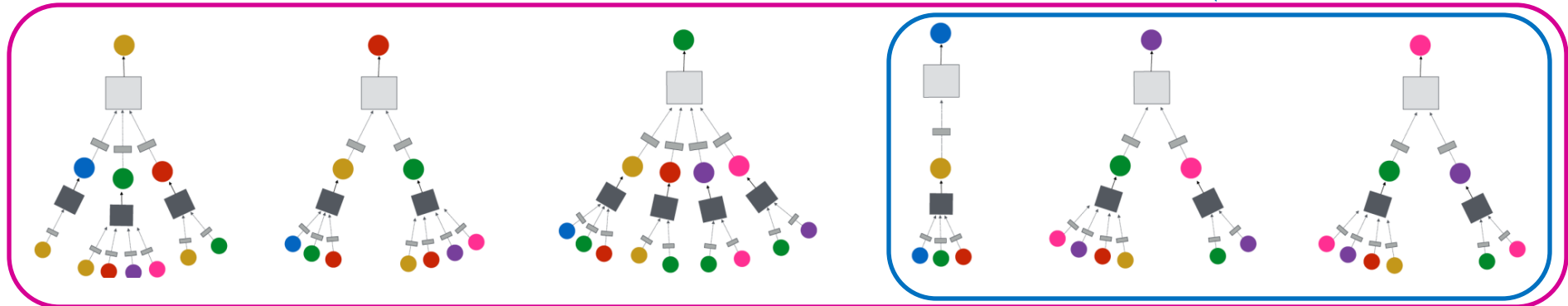
GCN Pipeline: Overview



INPUT GRAPH

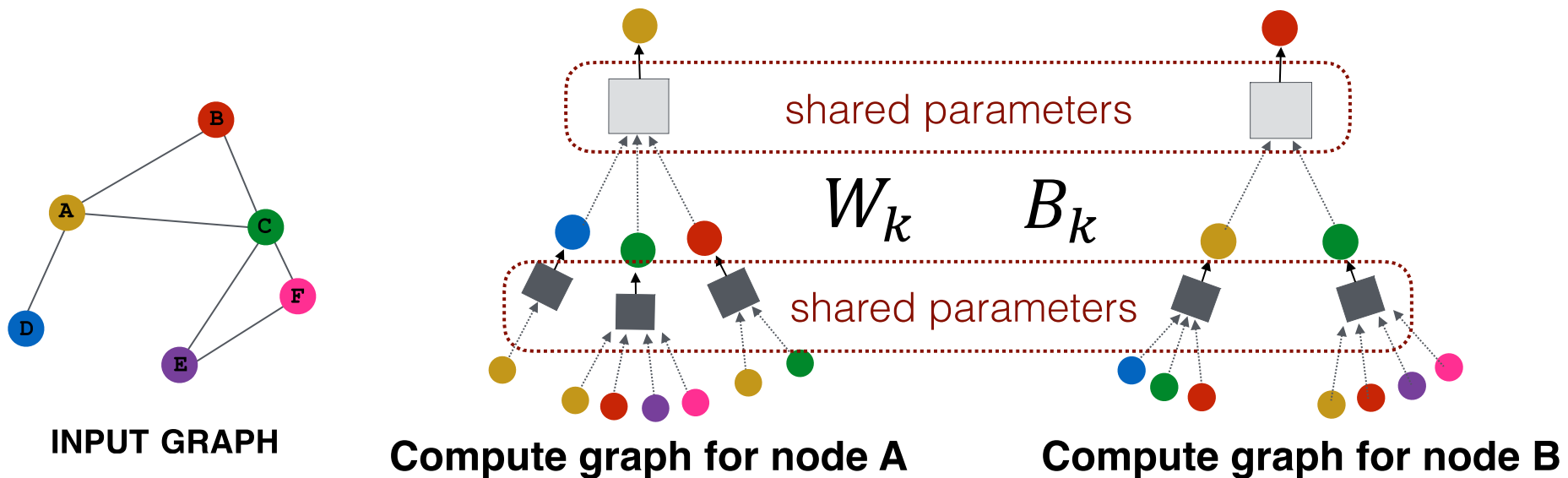
(4) Generate embeddings
for nodes as needed

Even for nodes we never
trained on!

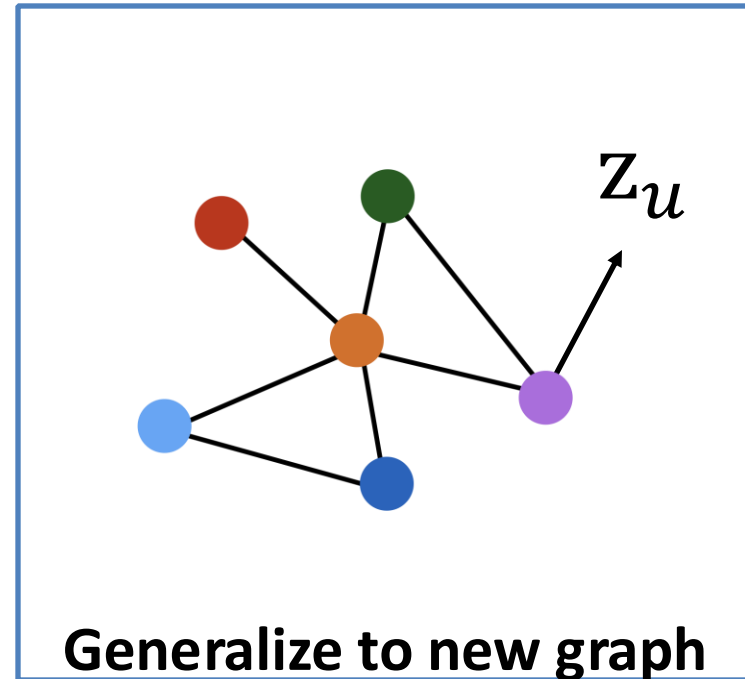
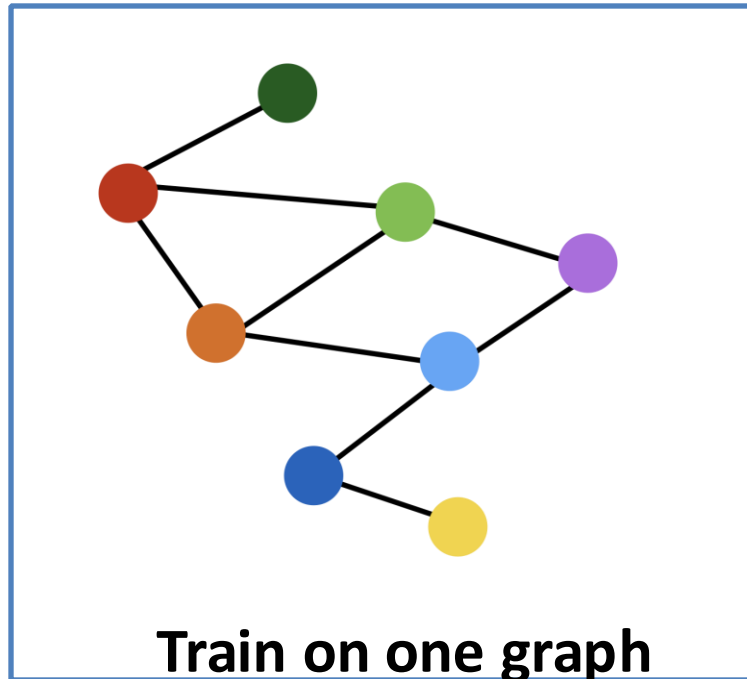


GCN is Inductive – Can Generalize to New Data

- The same aggregation parameters are **shared** for all nodes:
 - The number of model parameters is sublinear in $|V|$ and we can **generalize to unseen nodes!**



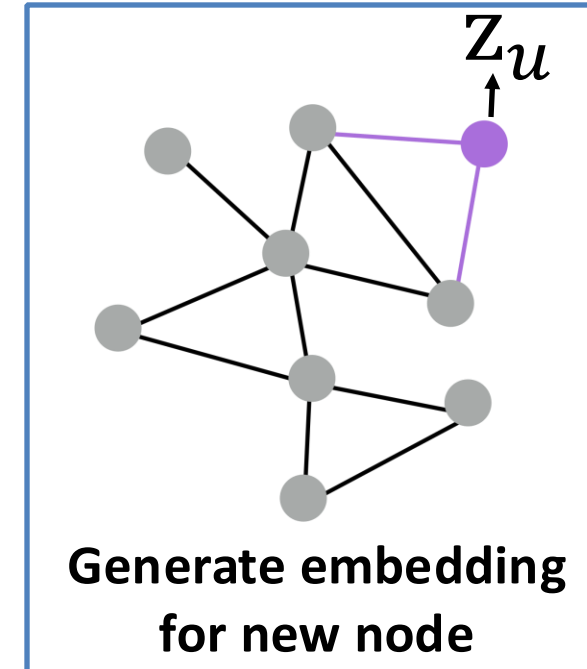
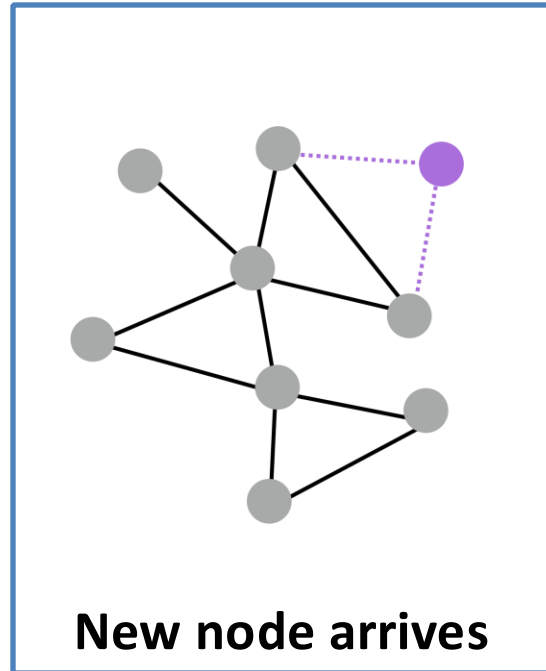
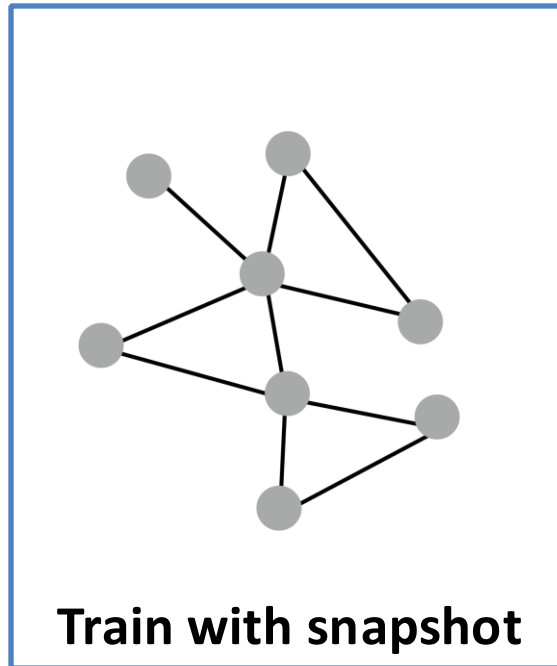
Inductive Capability: New Graphs



Inductive node embedding → Generalize to entirely unseen graphs

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Inductive Capability: New Nodes

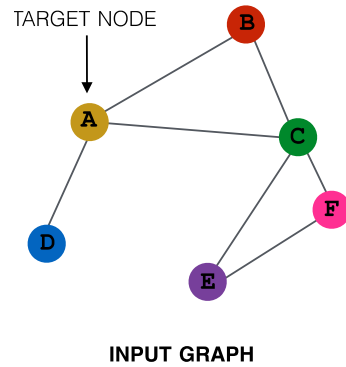


- Many application settings constantly encounter previously unseen nodes:
 - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

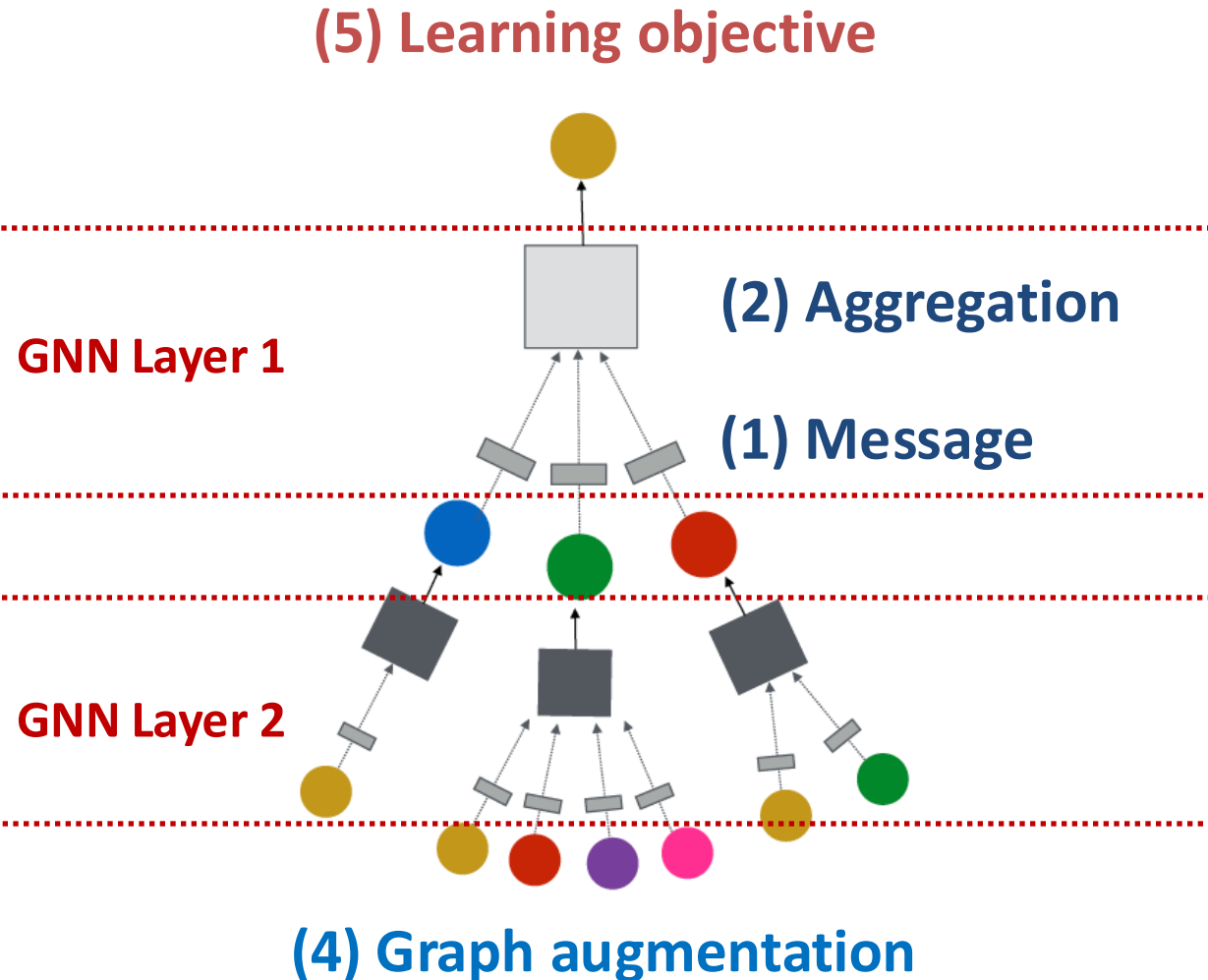
Graph Neural Networks

A General Perspective

A General GNN Framework



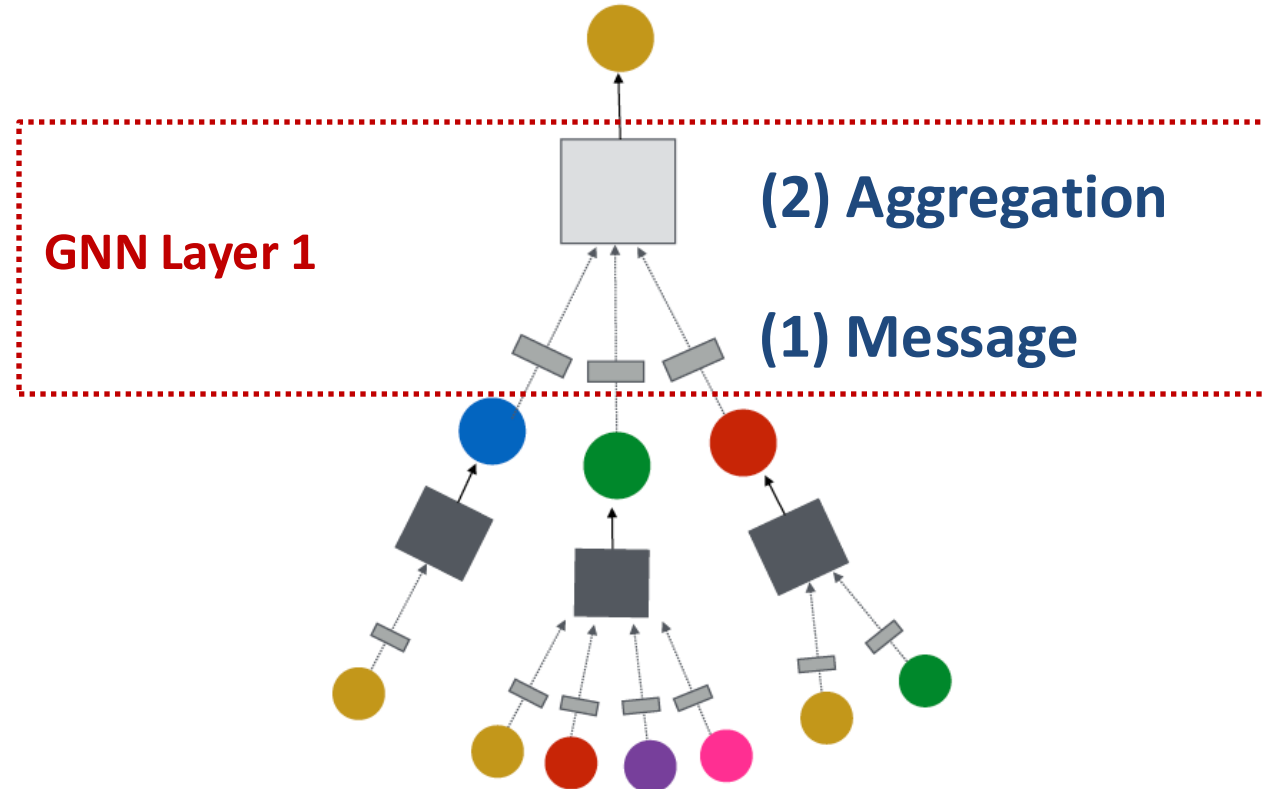
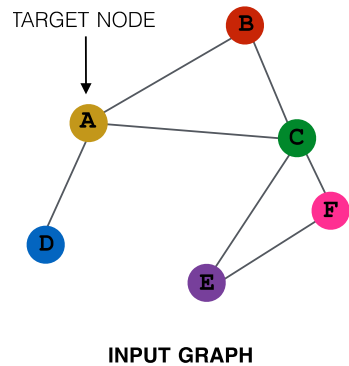
(3) Layer connectivity



A General GNN Framework (1)

GNN Layer = Message + Aggregation

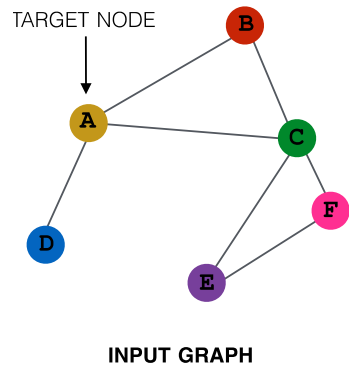
- Different instantiations under this perspective
- GCN, GraphSAGE, GAT, ...



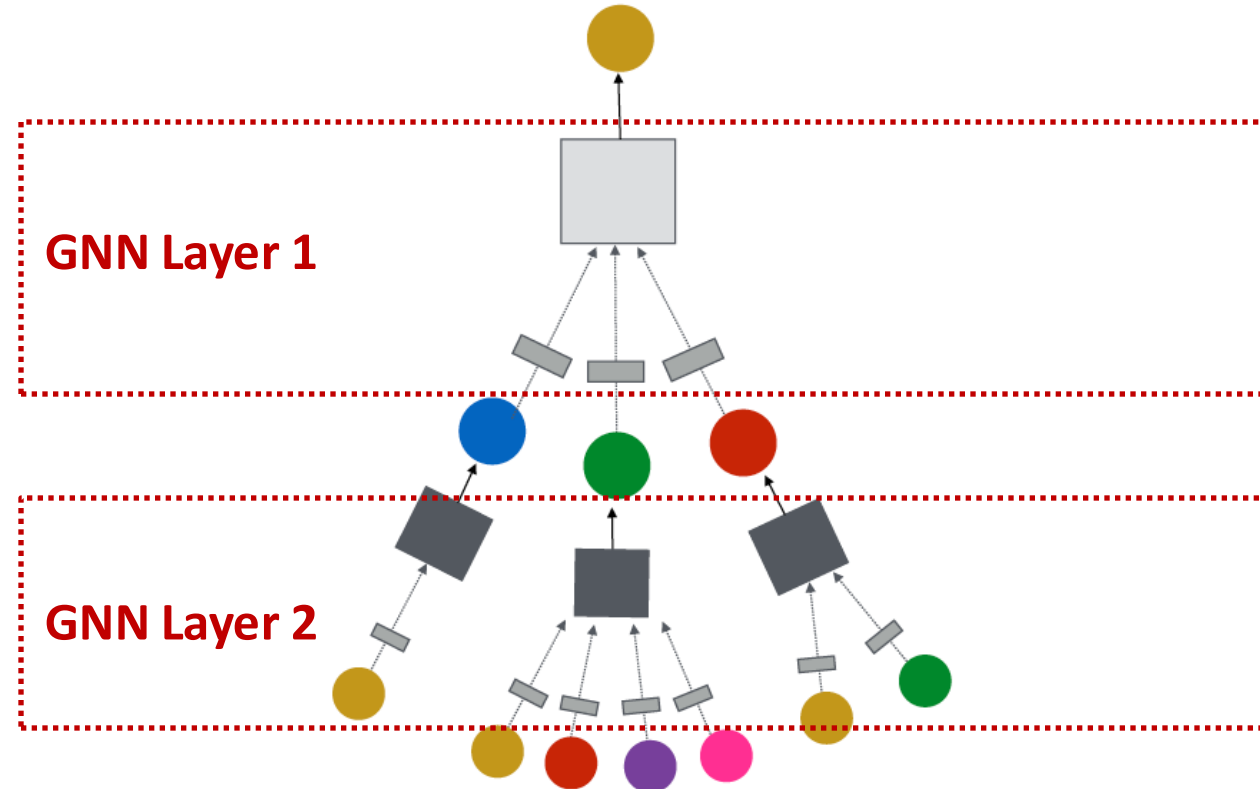
A General GNN Framework (2)

Connect GNN layers into a GNN

- Stack layers sequentially
- Ways of adding skip connections



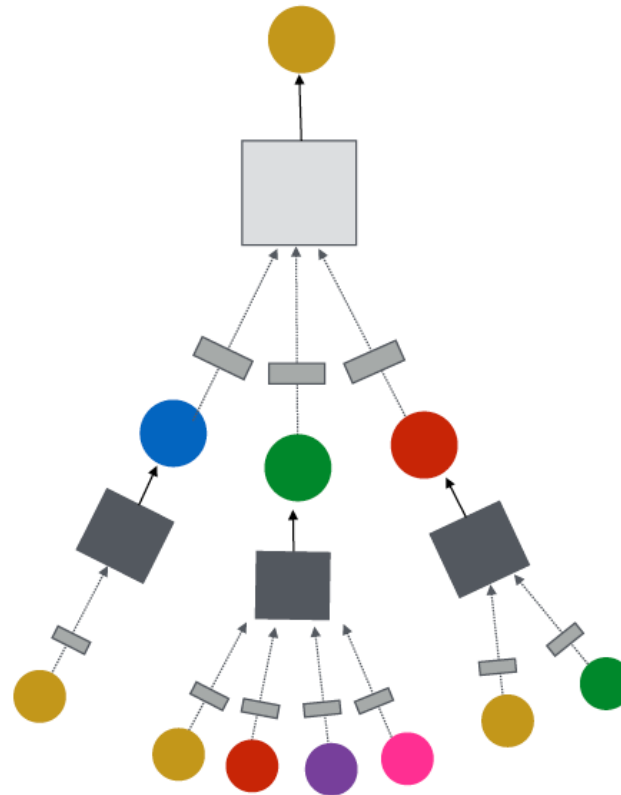
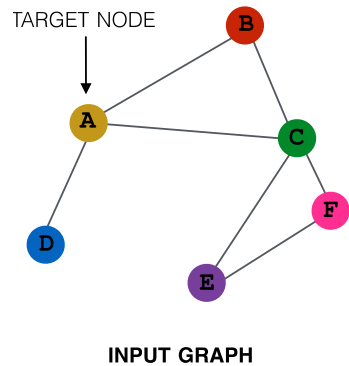
(3) Layer connectivity



A General GNN Framework (3)

Idea: Raw input graph $\neq$ computational graph

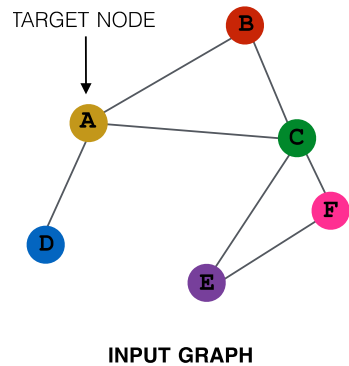
- Graph feature augmentation
- Graph structure augmentation



(4) Graph augmentation

A General GNN Framework (4)

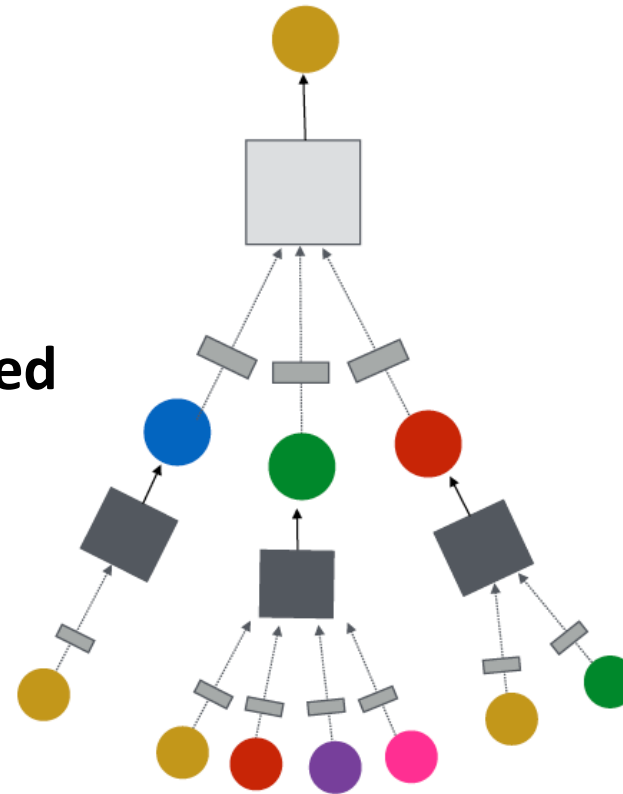
(5) Learning objective



How do we train a GNN

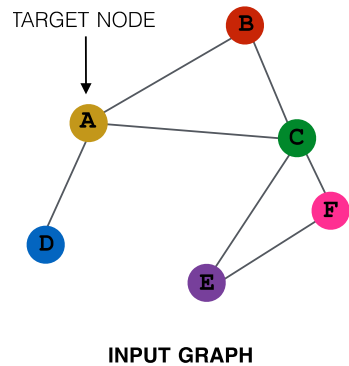
- Supervised/Unsupervised objectives
- Node/Edge/Graph level objectives

(We will discuss all of these later in class)

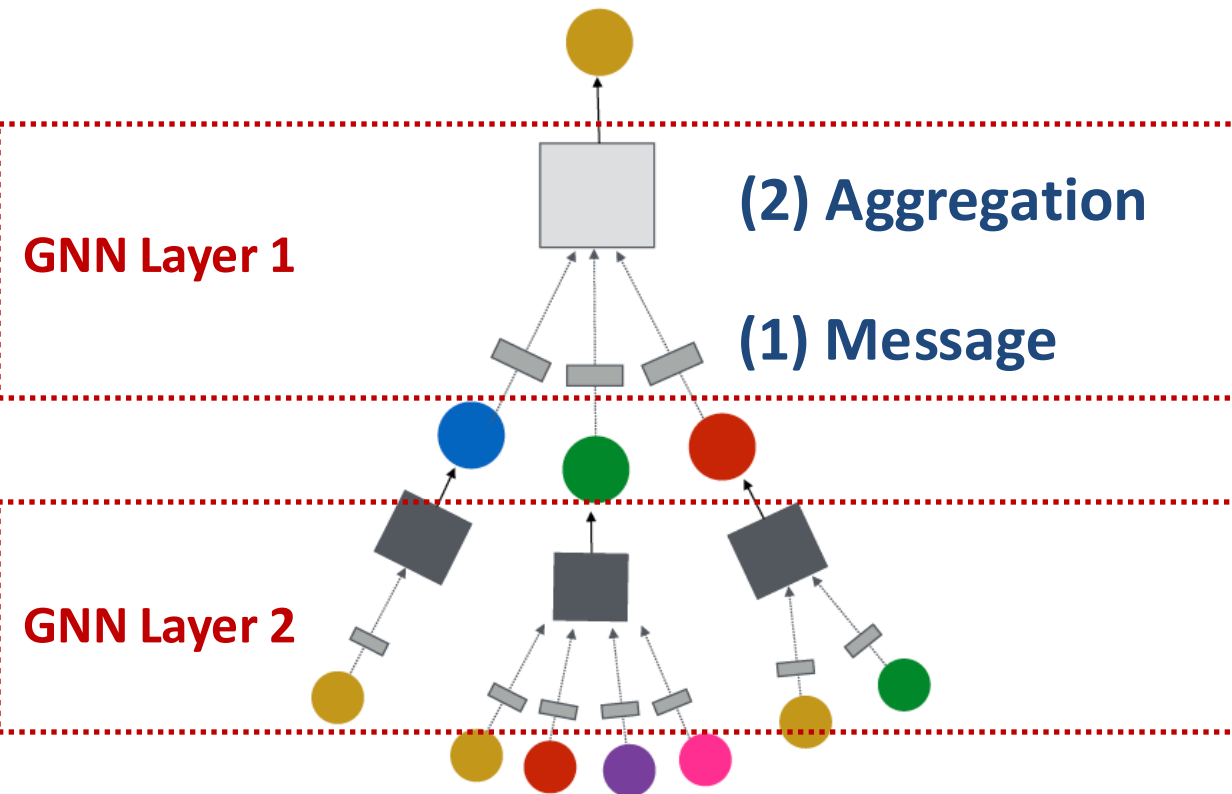


GNN Framework: Summary

(5) Learning objective



Layer
Connectivity



(4) Graph augmentation

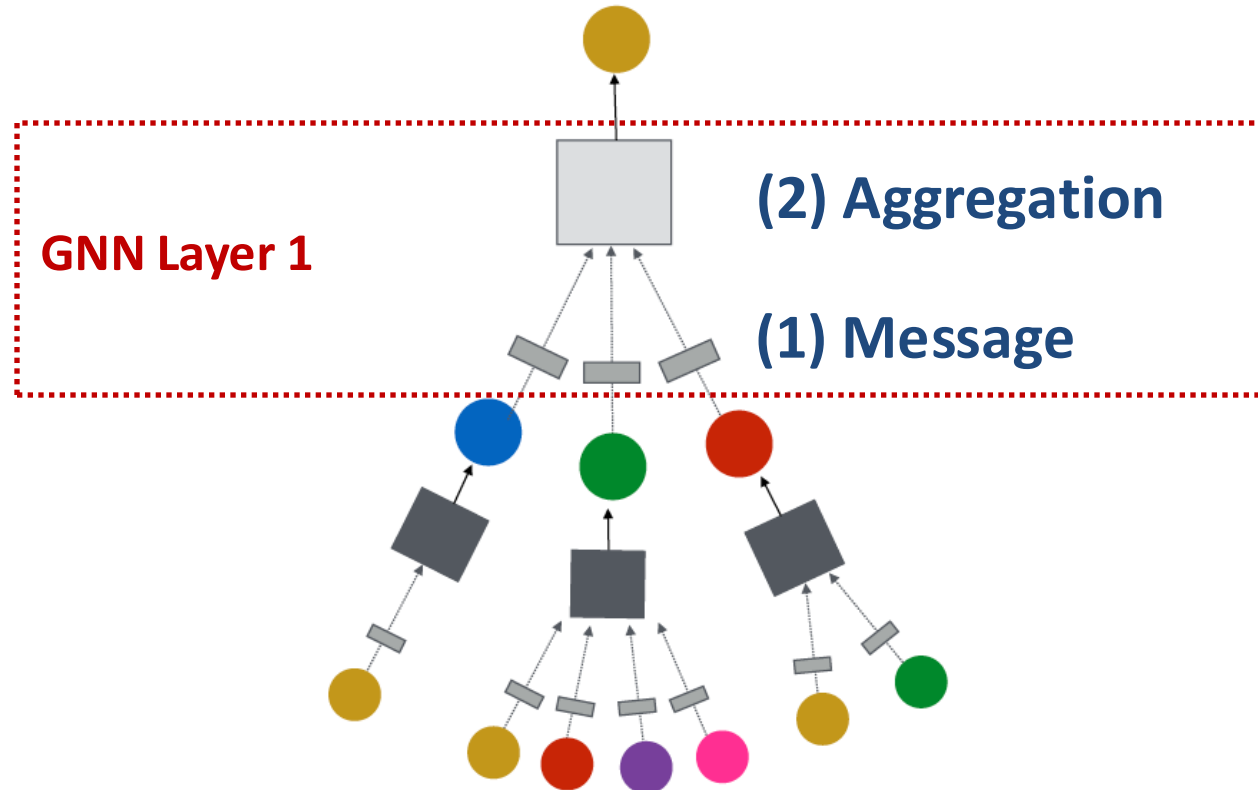
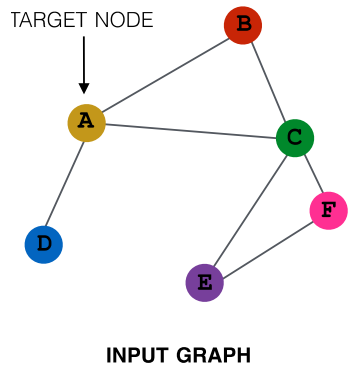
Graph Neural Networks: Model

A Single Layer of a GNN

A GNN Layer

GNN Layer = Message + Aggregation

- Different instantiations under this perspective
- GCN, GraphSAGE, GAT, ...



A Single GNN Layer

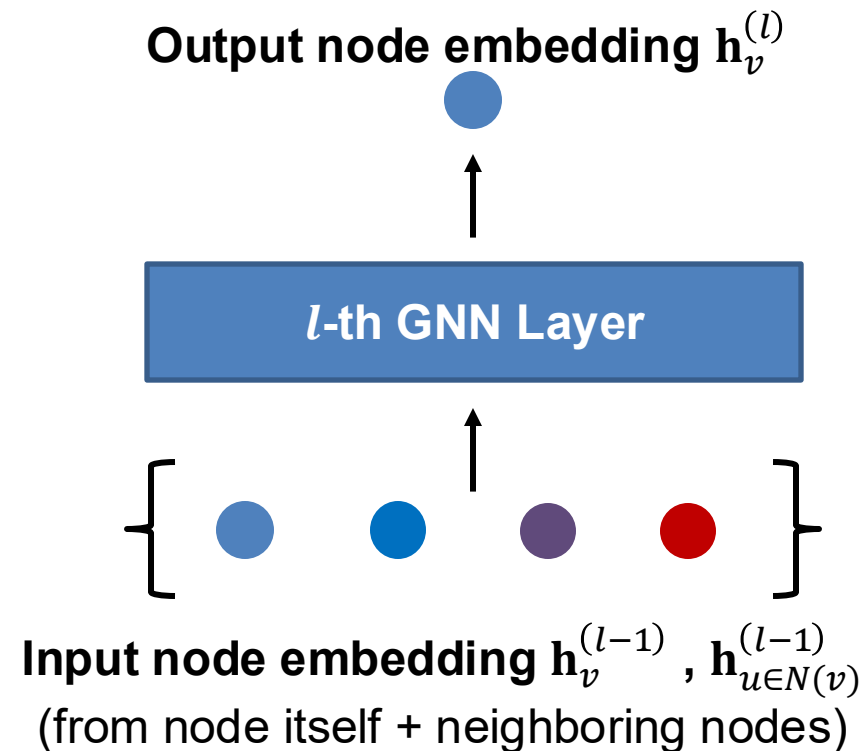
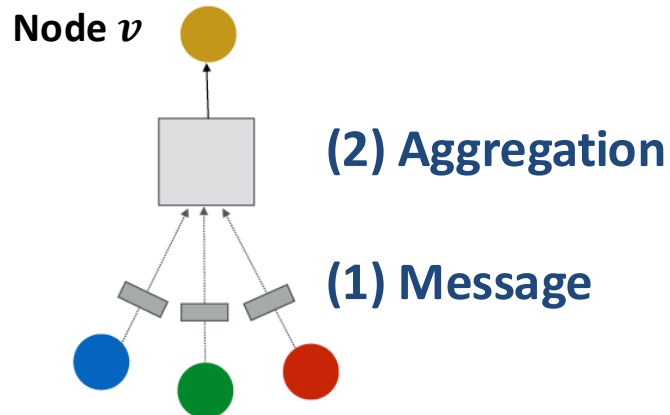
- **Idea of a GNN Layer:**

- Compress a set of vectors into a single vector

- **Two-step process:**

- (1) Message

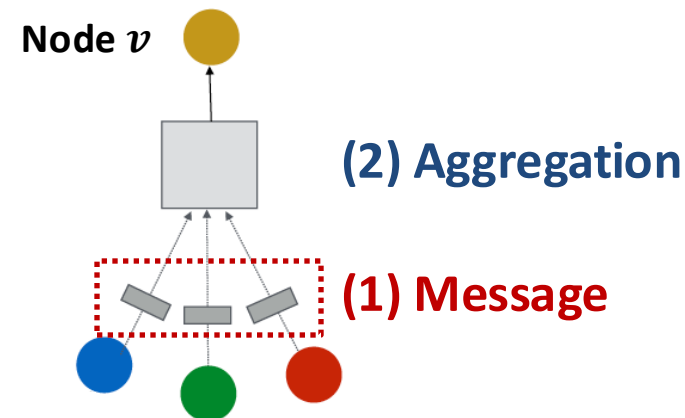
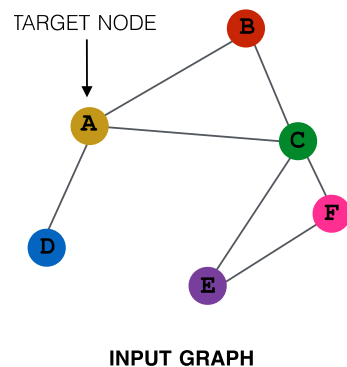
- (2) Aggregation



Message Computation

■ (1) Message computation

- **Message function:** $\mathbf{m}_u^{(l)} = \text{MSG}^{(l)}(\mathbf{h}_u^{(l-1)})$
 - **Intuition:** Each node will create a message, which will be sent to other nodes later
 - **Example:** A Linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$
 - Multiply node features with weight matrix $\mathbf{W}^{(l)}$



Message Aggregation

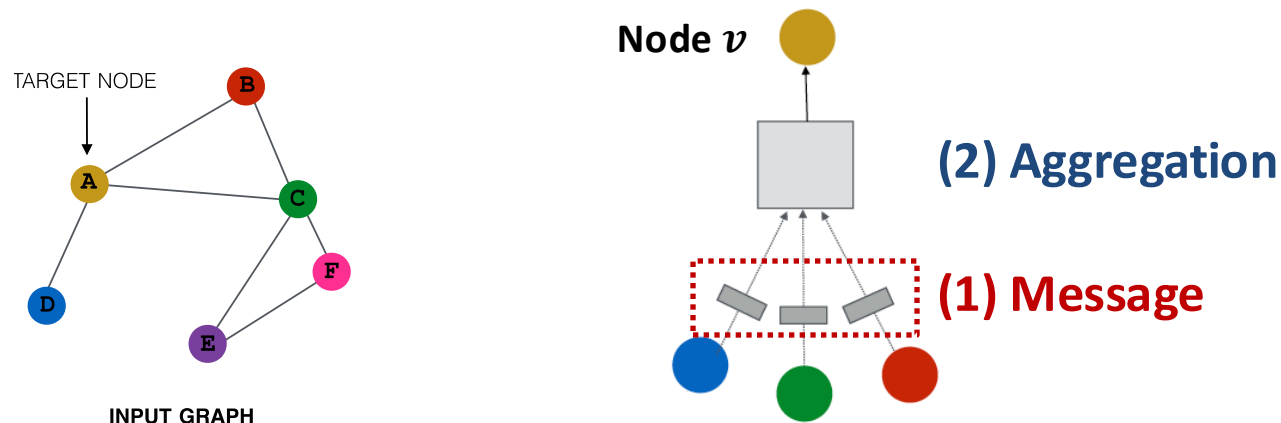
■ (2) Aggregation

- **Intuition:** Each node will aggregate the messages from node v 's neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$$

- **Example:** Sum($\cdot$), Mean($\cdot$) or Max($\cdot$) aggregator

- $\mathbf{h}_v^{(l)} = \text{Sum}(\{\mathbf{m}_u^{(l)}, u \in N(v)\})$



Message Aggregation: Issue

- **Issue:** Information from node v itself **could get lost**
 - Computation of $\mathbf{h}_v^{(l)}$ does not directly depend on $\mathbf{h}_v^{(l-1)}$
- **Solution:** Include $\mathbf{h}_v^{(l-1)}$ when computing $\mathbf{h}_v^{(l)}$
 - **(1) Message:** compute message from node v itself
 - Usually, a **different message computation** will be performed

$$\text{●} \text{●} \text{●} \quad \mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \quad \text{●} \quad \mathbf{m}_v^{(l)} = \mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)}$$

- **(2) Aggregation:** After aggregating from neighbors, we can **aggregate the message from node v itself**

- Via **concatenation or summation** **Then aggregate from node itself**

$$\mathbf{h}_v^{(l)} = \text{CONCAT} \left(\underbrace{\text{AGG} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)}_{\text{First aggregate from neighbors}}, \underbrace{\mathbf{m}_v^{(l)}}_{\text{Then aggregate from node itself}} \right)$$

A Single GNN Layer

- **Putting things together:**

- **(1) Message:** each node computes a message

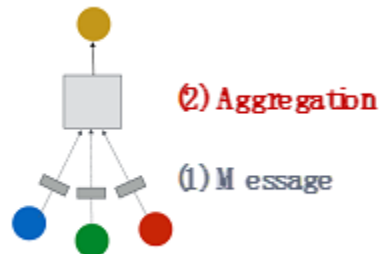
$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

- **(2) Aggregation:** aggregate messages from neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

- **Nonlinearity (activation):** Adds expressiveness

- Often written as $\sigma(\cdot)$: $\text{ReLU}(\cdot)$, $\text{Sigmoid}(\cdot)$, ...
- Can be added to **message or aggregation**



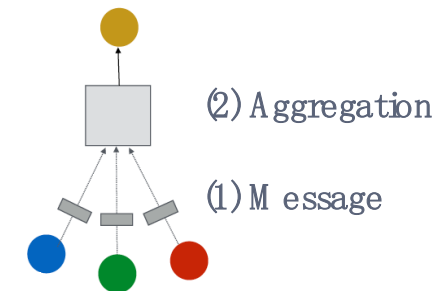
Classical GNN Layers: GCN (1)

- (1) Graph Convolutional Networks (GCN)

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \right)$$

- How to write this as Message + Aggregation?

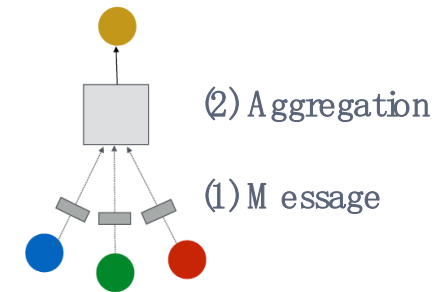
$$\mathbf{h}_v^{(l)} = \sigma \left(\underbrace{\sum_{u \in N(v)} \left(\underbrace{\mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}}_{\text{Message}} \right)}_{\text{Aggregation}} \right)$$



Classical GNN Layers: GCN (2)

■ (1) Graph Convolutional Networks (GCN)

$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|} \right)$$



■ Message:

- Each Neighbor: $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$

Normalized by node degree
(In the GCN paper they use a slightly different normalization)

■ Aggregation:

- **Sum** over messages + normalized by node degree $\Leftrightarrow$ **Mean** aggregation

- $\mathbf{h}_v^{(l)} = \sigma \left(\text{Mean} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right) \right)$

Classical GNN Layers: GraphSAGE

■ (2) GraphSAGE

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT} \left(\mathbf{h}_v^{(l-1)}, \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right) \right) \right)$$

■ How to write this as Message + Aggregation?

- Within the $\text{AGG}(\cdot)$: Message + Aggregation
- **Two-stage transformation:** Message + Aggregation + Message

- **Stage 1:** Message + Aggregation from node neighbors

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right)$$

- **Stage 2:** Further apply message transformation over node itself and its neighbors

$$\mathbf{h}_v^{(l)} \leftarrow \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)}) \right)$$

$$\text{Equivalent to: } \mathbf{h}_v^{(l)} \leftarrow \sigma \left(\mathbf{B}^{(l)} \mathbf{h}_v^{(l-1)} + \mathbf{W}^{(l)} \mathbf{h}_{N(v)}^{(l)} \right)$$

GraphSAGE Neighbor Aggregation

- **Mean:** Take a weighted average of neighbors

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$

Aggregation: Mean **Message: identity**

- **Pool:** Transform neighbor vectors and apply symmetric vector function
Mean($\cdot$) or Max($\cdot$)

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\})$$

Aggregation: Mean **Message: MLP**

- **LSTM:** Apply LSTM to reshuffled of neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))])$$

Aggregation: LSTM **Message: identity**

GraphSAGE: L2 Normalization

- ℓ_2 Normalization:

- **Optional:** Apply ℓ_2 normalization to $\mathbf{h}_v^{(l)}$ at every layer

- $$\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\|\mathbf{h}_v^{(l)}\|_2} \quad \forall v \in V \text{ where } \|u\|_2 = \sqrt{\sum_i u_i^2} \text{ } (\ell_2\text{-norm})$$

- Without ℓ_2 normalization, the embedding vectors have different scales (ℓ_2 -norm) for vectors
- In some cases (not always), normalization of embedding results in performance improvement
- After ℓ_2 normalization, all vectors will have the same ℓ_2 -norm

Classical GNN Layers: GAT (1)

■ (3) Graph Attention Networks

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \underbrace{\alpha_{vu}}_{\text{Attention weights}} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

■ In GCN / GraphSAGE

- $\alpha_{vu} = \frac{1}{|N(v)|}$ is the **weighting factor (importance)** of node u 's message to node v – Mean pooling
- $\Rightarrow \alpha_{vu}$ is defined **explicitly** based on the structural properties of the graph (node degree)
- $\Rightarrow$ All neighbors $u \in N(v)$ are equally important to node v

Classical GNN Layers: GAT (2)

■ (3) Graph Attention Networks

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Attention weights

Not all node's neighbors are equally important

- **Attention** is inspired by cognitive attention.
- The **attention** α_{vu} focuses on the important parts of the input data and fades out the rest.
 - **Idea:** the NN should devote more computing power on that small but important part of the data.
 - Which part of the data is more important depends on the context and is learned through training.

Graph Attention Networks

Can we do better than simple
neighborhood aggregation?

Can we let weighting factors α_{vu} to be
learned?

- **Goal:** Specify **arbitrary importance** to different neighbors of each node in the graph
- **Idea:** Compute embedding $\mathbf{h}_v^{(l)}$ of each node in the graph following an **attention strategy**:
 - Nodes attend over their neighborhoods' message
 - Implicitly specifying different weights to different nodes in a neighborhood

Attention Mechanism (1)

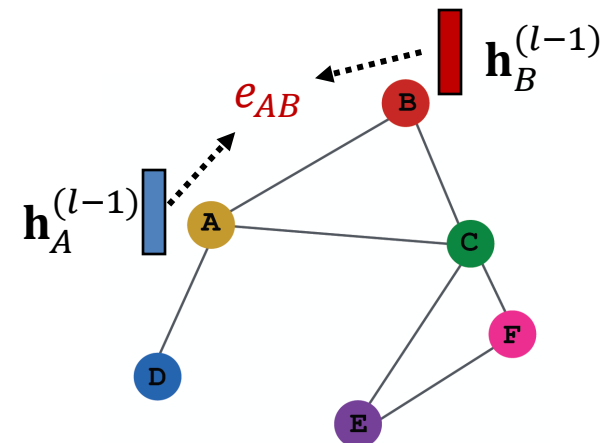
- Let α_{vu} be computed as a byproduct of an **attention mechanism** a :
 - (1) Let a compute **attention coefficients** e_{vu} across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_v^{(l-1)})$$

- e_{vu} indicates the importance of u 's message to node v - logits

Example:

$$e_{AB} = a(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)})$$



Attention Mechanism (2)

- **Normalize** e_{vu} into the **final attention weight** α_{vu}
 - Use the **softmax** function, so that $\sum_{u \in N(v)} \alpha_{vu} = 1$:

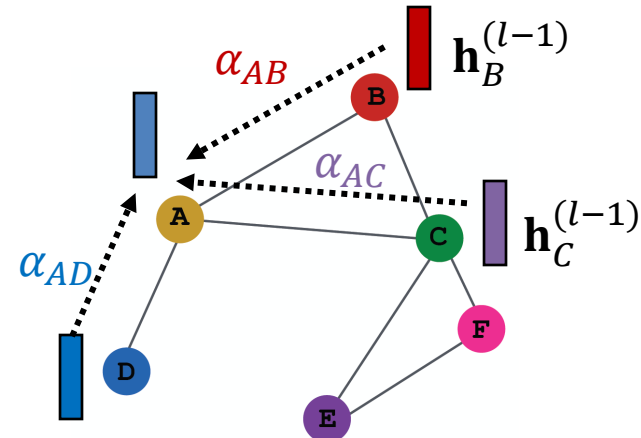
$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

- **Weighted sum** based on the **final attention weight** α_{vu}

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Weighted sum using α_{AB} , α_{AC} , α_{AD} :

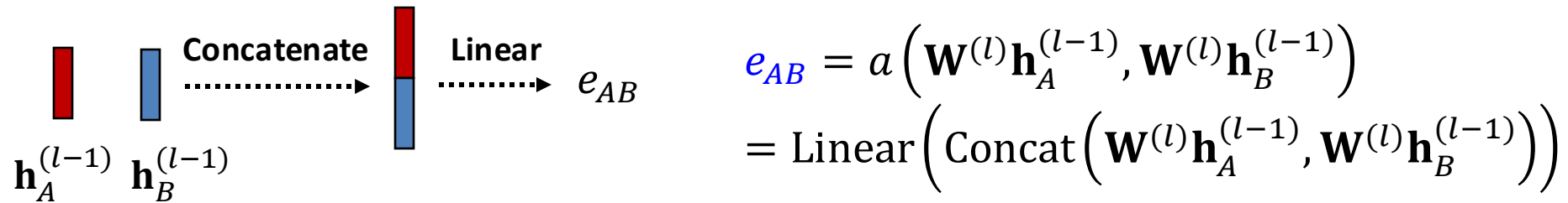
$$\mathbf{h}_A^{(l)} = \sigma\left(\alpha_{AB} \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} + \alpha_{AC} \mathbf{W}^{(l)} \mathbf{h}_C^{(l-1)} + \alpha_{AD} \mathbf{W}^{(l)} \mathbf{h}_D^{(l-1)}\right)$$



Attention Mechanism (3)

- What is the form of attention mechanism a ?

- The approach is agnostic to the choice of a
 - E.g., use a simple single-layer neural network
 - a have trainable parameters (weights in the Linear layer)



- Parameters of a are trained jointly:

- Learn the parameters together with weight matrices (i.e., other parameter of the neural net $\mathbf{W}^{(l)}$) in an end-to-end fashion

Attention Mechanism (4)

- **Multi-head attention:** Stabilizes the learning process of attention mechanism
 - Create **multiple attention scores** (each replica with a different set of parameters):

$$\mathbf{h}_v^{(l)}[1] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^1 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[2] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^2 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[3] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^3 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

- **Outputs are aggregated:**
 - By concatenation or summation
 - $\mathbf{h}_v^{(l)} = \text{AGG}(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3])$

Benefits of Attention Mechanism

- **Key benefit:** Allows for (implicitly) specifying **different importance values** (α_{vu}) **to different neighbors**
- **Storage efficient:**
 - Sparse matrix operations do not require more than $O(V + E)$ entries
 - **Fixed** number of parameters, irrespective of graph size
- **Localized -> computational efficiency:**
 - Only **attends over local network neighborhoods** (nodes that are direct neighbors)
 - GAT is a **sparse** attention (**Mask global attention with adjacency matrix**)
- **Inductive capability:**
 - It is a shared *edge-wise* mechanism
 - It does not depend on the global graph structure

Summary of the lecture

- **GCN Pipeline**
- **A general perspective for GNNs**
 - **GNN Layer:**
 - Transformation + Aggregation
 - Classic GNN layers: GCN, GraphSAGE, GAT
- **Next:** GNN layer connectivity, graph manipulation